Step 4. Performance Report: Query Optimization with Indexing

Database: `bookstore`

Collection: `books`

Target Query: `db.books.find({ category: "Programming", published_year: { $gte: 2020 } })`

1. Why Indexes Improve Performance

In a database without indexes, documents are stored in an unordered sequence. When running a query with specific filters, the database engine has to inspect every single document to check if it matches the criteria.

An **index** works like an alphabetical index at the back of a textbook. It creates a separate, highly organized data structure (usually a B-Tree) that holds pointers to the actual documents. Instead of scanning the whole collection, MongoDB can instantly traverse the index tree to pinpoint the exact location of the requested data, dramatically reducing disk I/O and CPU usage.

2. Differences Between COLLSCAN and IXSCAN

Database execution plans reveal how MongoDB executes a query. The two major scan strategies observed in this lab are:

- `COLLSCAN` **(Collection Scan):** The database performs a full sequential scan of the entire collection from start to finish. It is highly inefficient because its time complexity is $O(N)$, meaning the workload grows linearly with the number of documents.
- `IXSCAN` **(Index Scan):** The database queries the index structure first. It allows the engine to skip unnecessary data entirely and jump directly to the matched index keys. This stage is followed by a `FETCH` operation, which retrieves the actual full documents from memory based on the index pointers.

3. Performance Metrics: Before vs. After Indexing

The compound index was successfully created using the following command:

JavaScript

```
db.books.createIndex({ category: 1, published_year: 1 })
```

Based on the `explain("executionStats")` output, here is the direct comparison of the execution metrics:

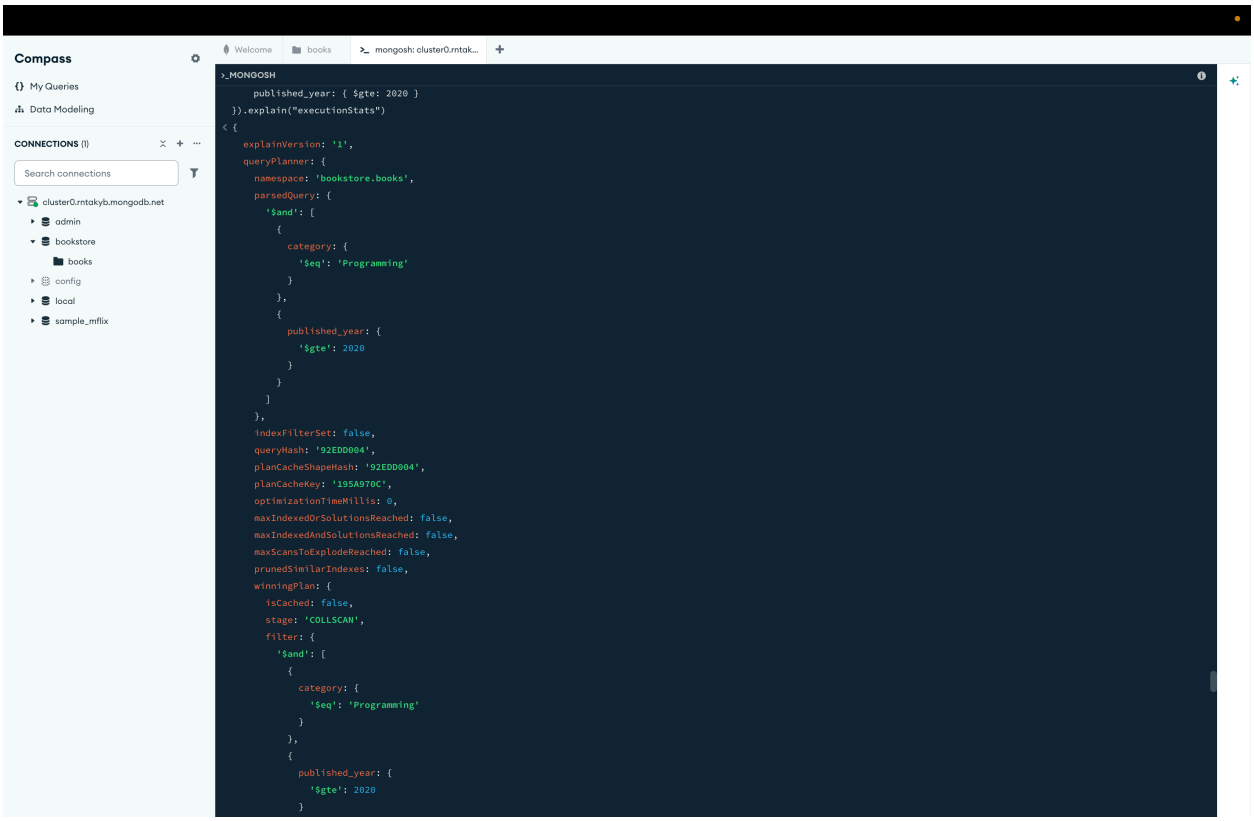
Performance Metric	Before Indexing (COLLSCAN)	After Indexing (IXSCAN)
Winning Plan Stage	`COLLSCAN`	`IXSCAN`
Total Keys Examined	`0`	`1`
Total Documents Examined	`33`	`1`
Documents Returned (`nReturned`)	`1`	`1`
Execution Time (`executionTimeMillis`)	`0 ms`	`1 ms`

Key Analysis:

- **Documents Examined:** Before indexing, MongoDB had to scan **all 33 documents** in the collection to find just 1 match. After building the compound index, the database examined **exactly 1 document** — a 100% precision rate.
- **Execution Time:** In a tiny sandbox environment containing only 33 entries, the execution time sits within the margin of error (`0 ms` vs `1 ms`). However, the metric that truly defines performance scalability is `totalDocsExamined`. On a production database with millions of records, a `COLLSCAN` would cause severe bottlenecks (seconds or minutes of execution time), whereas the `IXSCAN` would keep the response time near 0 milliseconds.

Screenshots:

Before:



the plan:

```
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'bookstore.books',
    parsedQuery: {
      '$and': [
        {
          category: {
            '$eq': 'Programming'
          }
        },
        {
          published_year: {
            '$gte': 2020
```

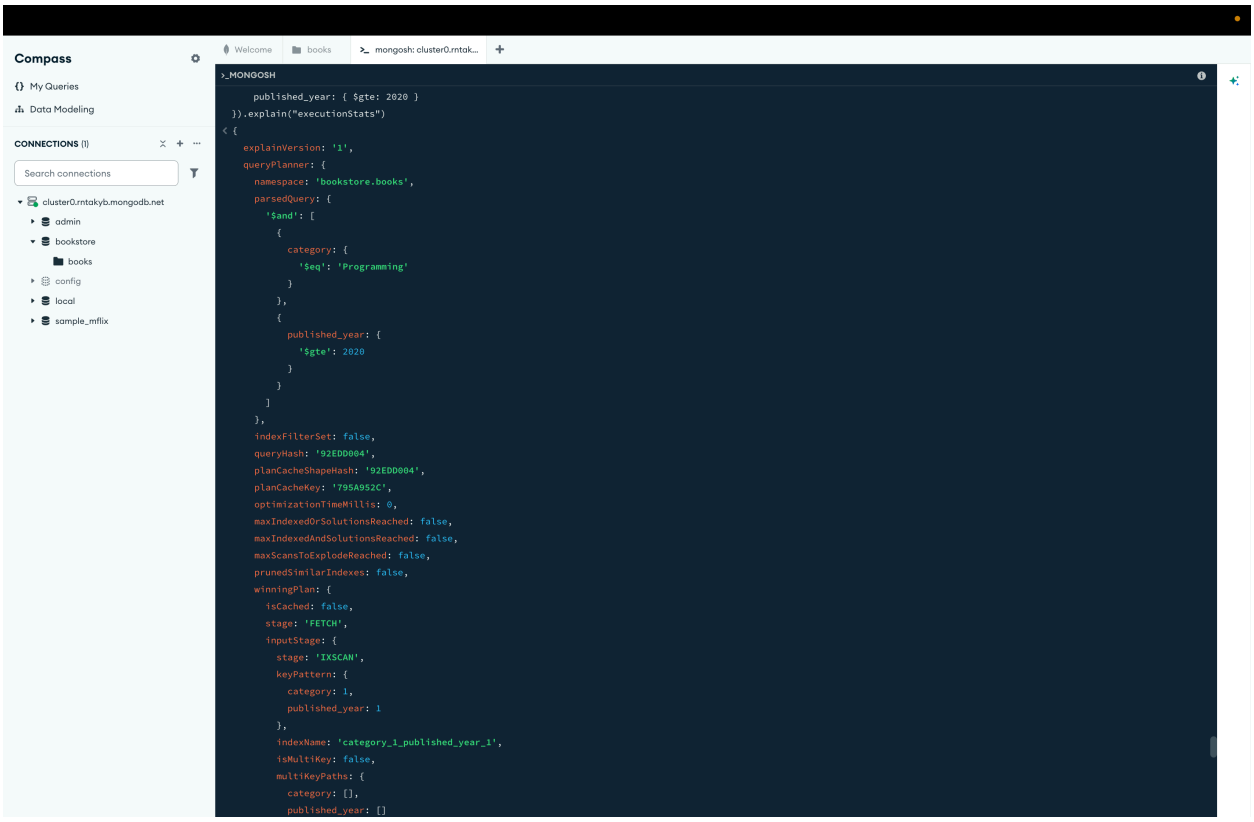
```
            }
          }
        ]
      },
      indexFilterSet: false,
      queryHash: '92EDD004',
      planCacheShapeHash: '92EDD004',
      planCacheKey: '195A970C',
      optimizationTimeMillis: 0,
      maxIndexedOrSolutionsReached: false,
      maxIndexedAndSolutionsReached: false,
      maxScansToExplodeReached: false,
      prunedSimilarIndexes: false,
      winningPlan: {
        isCached: false,
        stage: 'COLLSCAN',
        filter: {
          '$and': [
            {
              category: {
                '$eq': 'Programming'
              }
            },
            {
              published_year: {
                '$gte': 2020
              }
            }
          ]
        },
        direction: 'forward'
      },
      rejectedPlans: []
    },
    executionStats: {
      executionSuccess: true,
```

```
    nReturned: 1,
    executionTimeMillis: 0,
    totalKeysExamined: 0,
    totalDocsExamined: 33,
    executionStages: {
      isCached: false,
      stage: 'COLLSCAN',
      filter: {
        '$and': [
          {
            category: {
              '$eq': 'Programming'
            }
          },
          {
            published_year: {
              '$gte': 2020
            }
          }
        ]
      },
      nReturned: 1,
      executionTimeMillisEstimate: 0,
      works: 34,
      advanced: 1,
      needTime: 32,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
      direction: 'forward',
      docsExamined: 33
    }
  },
  queryShapeHash: 'BD5A3F1E50B6E3480036559A9D1E7891433029347A
12443523DD94FE50286C1E',
```

```
  command: {
    find: 'books',
    filter: {
      category: 'Programming',
      published_year: {
        '$gte': 2020
      }
    },
    '$db': 'bookstore'
  },
  serverInfo: {
    host: 'ac-unlpuj7-shard-00-01.rntakyb.mongodb.net',
    port: 27017,
    version: '8.0.27',
    gitVersion: 'a2b88eae13acf01bc0081c8e1dbc6b1dd3aaeeb6'
  },
  serverParameters: {
    internalQueryFacetBufferSizeBytes: 104857600,
    internalQueryFacetMaxOutputDocSizeBytes: 104857600,
    internalLookupStageIntermediateDocumentMaxSizeBytes: 1679
3600,
    internalDocumentSourceGroupMaxMemoryBytes: 104857600,
    internalQueryMaxBlockingSortMemoryUsageBytes: 33554432,
    internalQueryProhibitBlockingMergeOnMongoS: 0,
    internalQueryMaxAddToSetBytes: 104857600,
    internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 1048
57600,
    internalQueryFrameworkControl: 'trySbeRestricted',
    internalQueryPlannerIgnoreIndexWithCollationForRegex: 1
  },
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1783861498, i: 9 }),
    signature: {
      hash: Binary.createFromBase64('u5LzyvWDTCz8o/i96wrm5UCN
qqQ=', 0),
```

```
      keyId: 7624585682282873000
    }
  },
  operationTime: Timestamp({ t: 1783861498, i: 9 })
}
```

After:



the plan:

```
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'bookstore.books',
    parsedQuery: {
```

```
        '$and': [
          {
            category: {
              '$eq': 'Programming'
            }
          },
          {
            published_year: {
              '$gte': 2020
            }
          }
        ]
      },
      indexFilterSet: false,
      queryHash: '92EDD004',
      planCacheShapeHash: '92EDD004',
      planCacheKey: '795A952C',
      optimizationTimeMillis: 0,
      maxIndexedOrSolutionsReached: false,
      maxIndexedAndSolutionsReached: false,
      maxScansToExplodeReached: false,
      prunedSimilarIndexes: false,
      winningPlan: {
        isCached: false,
        stage: 'FETCH',
        inputStage: {
          stage: 'IXSCAN',
          keyPattern: {
            category: 1,
            published_year: 1
          },
          indexName: 'category_1_published_year_1',
          isMultiKey: false,
          multiKeyPaths: {
            category: [],
            published_year: []
```

```
        },
        isUnique: false,
        isSparse: false,
        isPartial: false,
        indexVersion: 2,
        direction: 'forward',
        indexBounds: {
          category: [
            '["Programming", "Programming"]'
          ],
          published_year: [
            '[2020, inf.0]'
          ]
        }
      }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 1,
    executionTimeMillis: 1,
    totalKeysExamined: 1,
    totalDocsExamined: 1,
    executionStages: {
      isCached: false,
      stage: 'FETCH',
      nReturned: 1,
      executionTimeMillisEstimate: 0,
      works: 2,
      advanced: 1,
      needTime: 0,
      needYield: 0,
      saveState: 0,
      restoreState: 0,
      isEOF: 1,
```

```
      docsExamined: 1,
      alreadyHasObj: 0,
      inputStage: {
        stage: 'IXSCAN',
        nReturned: 1,
        executionTimeMillisEstimate: 0,
        works: 2,
        advanced: 1,
        needTime: 0,
        needYield: 0,
        saveState: 0,
        restoreState: 0,
        isEOF: 1,
        keyPattern: {
          category: 1,
          published_year: 1
        },
        indexName: 'category_1_published_year_1',
        isMultiKey: false,
        multiKeyPaths: {
          category: [],
          published_year: []
        },
        isUnique: false,
        isSparse: false,
        isPartial: false,
        indexVersion: 2,
        direction: 'forward',
        indexBounds: {
          category: [
            '["Programming", "Programming"]'
          ],
          published_year: [
            '[2020, inf.0]'
          ]
        },
```

```
        keysExamined: 1,
        seeks: 1,
        dupsTested: 0,
        dupsDropped: 0
      }
    }
  },
  queryShapeHash: 'BD5A3F1E50B6E3480036559A9D1E7891433029347A
12443523DD94FE50286C1E',
  command: {
    find: 'books',
    filter: {
      category: 'Programming',
      published_year: {
        '$gte': 2020
      }
    },
    '$db': 'bookstore'
  },
  serverInfo: {
    host: 'ac-unlpuj7-shard-00-01.rntakyb.mongodb.net',
    port: 27017,
    version: '8.0.27',
    gitVersion: 'a2b88eae13acf01bc0081c8e1dbc6b1dd3aaeeb6'
  },
  serverParameters: {
    internalQueryFacetBufferSizeBytes: 104857600,
    internalQueryFacetMaxOutputDocSizeBytes: 104857600,
    internalLookupStageIntermediateDocumentMaxSizeBytes: 1679
3600,
    internalDocumentSourceGroupMaxMemoryBytes: 104857600,
    internalQueryMaxBlockingSortMemoryUsageBytes: 33554432,
    internalQueryProhibitBlockingMergeOnMongoS: 0,
    internalQueryMaxAddToSetBytes: 104857600,
    internalDocumentSourceSetWindowFieldsMaxMemoryBytes: 1048
57600,
```

```
    internalQueryFrameworkControl: 'trySbeRestricted',
    internalQueryPlannerIgnoreIndexWithCollationForRegex: 1
  },
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1783861985, i: 15 }),
    signature: {
      hash: Binary.createFromBase64('pfY6lOpM5brNE7rypTFfl8ls
ERs=', 0),
      keyId: 7624585682282873000
    }
  },
  operationTime: Timestamp({ t: 1783861985, i: 15 })
}
```